



Mauritania Real Estate Recommendation System

Complete Technical Documentation
For Beginners — From Zero to Full Understanding

Built with: Python · Streamlit · TF-IDF · Mistral AI · Google STT · edge-tts

Dataset: voursa.com — Mauritania's Largest Property Marketplace

Author: aliywedad · Date: June 2026

Table of Contents

- 1.** Project Overview — What Is Immo.AI?
- 2.** Project Architecture — How All Pieces Connect
- 3.** The Dataset — Where The Data Comes From
- 4.** Data Preprocessing — Cleaning Raw Data
- 5.** Feature Engineering — Making Data Smarter
- 6.** The Recommender System Algorithm
 - 6a.** Why Content-Based Filtering?
 - 6b.** Algorithms We DID NOT Use & Why
 - 6c.** TF-IDF: The Text Similarity Engine
 - 6d.** Hybrid Scoring Formula
- 7.** Voice to Text — How The Microphone Works
- 8.** Text to Speech — How The AI Speaks Back
- 9.** The AI Brain — Mistral Large & JARVIS
- 10.** The Web App — Streamlit UI
- 11.** Evaluation — How We Measure Quality
- 12.** Future Improvements
- 13.** Glossary — Key Terms Explained

1. Project Overview — What Is Immo.AI?

Immo.AI is an intelligent real estate assistant for **Mauritania**. Think of it as a smart helper that you can **talk to** — using your voice or keyboard — and it will find property listings that match what you are looking for, from a database of over 1,852 real properties scraped from **voursa.com**, Mauritania's largest online marketplace.

Imagine you say: *"Hey Jarvis, find me a villa in Tévragh Zeina under 2 million MRU"* — the system will:

- Convert your voice to text using Google's Speech Recognition
- Send that text to Mistral AI (like ChatGPT but by a French company)
- Mistral extracts: type=villa, location=Tévragh Zeina, max_price=2,000,000
- The recommender searches the database using TF-IDF + Hybrid Scoring
- Top results are formatted by Mistral and spoken back to you using neural TTS

■ *This is a FULL AI pipeline: Voice → Text → NLP Understanding → Smart Search → Results → Voice response. All in real-time.*

2. Project Architecture — How All Pieces Connect

Below is the complete data flow. Think of it like an assembly line where each step transforms the user's request into a useful answer.



File Structure

File / Folder	Role	Key Functions
app.py	Main Streamlit web application — UI, tabs, layout	get_data(), get_recommender(), render_property_card()
chatbot.py	Original CLI prototype (voice chatbot without recommender)	speak(), listen(), chat()

e-commerce-mr.csv	Raw dataset — 1,852+ property listings from voursa.com	Input data
src/data_cleaning.py	Cleans and normalises the raw CSV	_parse_price(), _parse_days_ago(), _main_location(), load_and_clean()
src/recommender.py	Core ML engine — TF-IDF + Hybrid Scoring	engineer_features(), Recommender.fit(), recommend(), evaluate()
src/mistral_client.py	Mistral AI integration — JARVIS personality, search trigger, result formatting	chat_turn(), format_results()
src/voice.py	Speech-to-Text + Text-to-Speech	transcribe_audio_bytes(), text_to_speech_bytes(), audio_hash()
requirements.txt	Python package dependencies list	—
.env	Secret API keys (never commit to git)	MISTRAL_API_KEY

3. The Dataset — Where The Data Comes From

The dataset is stored in **e-commerce-mr.csv** — a ~1.7 MB CSV file containing property listings scraped from **voursa.com**, Mauritania's main classifieds website.

Dataset Statistics

Metric	Value
Total raw rows (approx)	~2,000+
After cleaning (real estate)	1,852 listings
Categories	real_estate, vehicles, electronics, etc.
Locations covered	Nouakchott districts + other Mauritanian cities
Price range (after cleaning)	~50,000 — 50,000,000 MRU
Time period	Listings from past days to years ago

Raw CSV Columns

Column Name	What It Contains	Example Value
Title	Property listing headline	"Villa F5 Tevragh Zeina avec jardin"
Price	Raw price string with currency	"1,500,000 MRU"
Location+Name	Full location text (renamed to Location)	"Tevragh Zeina, Nouakchott"
Date_Posted	Relative date string	"3 days ago" / "2 months ago"
Views	Number of times the listing was viewed	"842"
Status	Listing availability	"Available" / "Sold Out"
Category	Type of listing	"real_estate"
Link	URL to the listing on voursa.com	https://voursa.com/...
Image	URL of the listing photo	https://...jpg

4. Data Preprocessing — Cleaning Raw Data

Raw data from real websites is **messy**. Prices have commas and currency symbols. Locations are inconsistently formatted. Some entries are duplicates or have nonsensical values. Before we can use any algorithm, we must clean the data. This is done in **src/data_cleaning.py**.

Step 1: Price Parsing — `_parse_price()`

The Price column contains strings like *"1,500,000 MRU"*. We need a pure number. The function uses **regex** to strip every character that is NOT a digit, converting *"1,500,000 MRU"* → **1500000.0**. If the resulting number is ≤ 1 , we treat it as invalid (placeholder entry) and discard it.

```
def _parse_price(raw):
    digits = re.sub(r'[^\d]', '', str(raw)) # Remove non-digits
    val = float(digits)
    return None if val <= 1 else val # Discard junk values
```

Step 2: Date Parsing — `_parse_days_ago()`

Dates come as human-readable strings: *"3 days ago"*, *"2 months ago"*, *"1 year ago"*. We convert these to an integer number of days so we can compute freshness mathematically.

```
# '3 days ago'    → 3
# '2 months ago' → 60 (2 × 30)
# '1 year ago'   → 365
# '5 hours ago'  → 0 (same day)
```

Step 3: Location Normalisation — `_main_location()`

Location strings vary wildly: *"Tevragh Zeina, Nouakchott, Mauritania"*, *"Tev. Zeina"*, etc. We match against a hard-coded list of known Mauritanian neighborhoods (Tevragh Zeina, Ksar, Arafat, Teyarett, Dar Naim, Toujounine, Sebkha, El Mina, Riyadh, Nouadhibou, etc.) and return the canonical name. If no match, we take the first word.

Step 4: Removing Invalid Rows

- Drop rows with no price (Price_MRU is NaN) or empty Title
- Strip whitespace from Title and remove empty-string titles

Step 5: Outlier Removal (Per-Category Percentile Clipping)

Some listings have absurd prices — a typo turning 1,500,000 into 1,500,000,000. These outliers would break our scoring. We compute the **1st and 99th percentile** of prices **per category** and discard everything outside that range.

```
for cat, group in df.groupby('Category'):
    lo = group['Price_MRU'].quantile(0.01)
    hi = group['Price_MRU'].quantile(0.99)
    # Keep only rows within [lo, hi]
```

Step 6: Deduplication

If two rows have the same Title + Price + Main_Location, they are the same listing posted twice. We keep only one copy using pandas **drop_duplicates**.

Step 7: Views Normalisation

The Views column sometimes contains text or missing values. We convert it to a number, replacing errors with 0: `pd.to_numeric(df['Views'], errors='coerce').fillna(0)`

■ *After all cleaning steps, we have a clean, reliable dataset ready for feature engineering and recommendation.*

5. Feature Engineering — Making Data Smarter

Raw data has Title and Price. But our algorithm needs **numerical vectors** (lists of numbers) to compute similarity. Feature engineering transforms raw columns into meaningful numbers. This is done in `src/recommender.py` → `engineer_features()`.

Feature 1: Price Bucket (Categorical Label)

We divide all prices into 5 equal-sized groups using **quantile cuts** (`pd.qcut`). Each group gets a label:

Bucket	Meaning	Price Range (approx)
budget	Cheapest 20% of listings	< 200,000 MRU
affordable	Next 20%	200K – 500K MRU
mid-range	Middle 20%	500K – 1.5M MRU
premium	Upper 20%	1.5M – 5M MRU
luxury	Most expensive 20%	> 5M MRU

This label is then **appended to the text** that gets vectorised, so a query for "cheap apartment" will match budget/affordable listings better.

Feature 2: The Combined Text Field (`_text`)

We concatenate multiple columns into a single text field:

```
df['_text'] = (Title + ' ' + Location + ' ' +  
               + Main_Location + ' ' + price_bucket)
```

This single text field is what the TF-IDF vectoriser reads. It combines the property name, location details, and price category into one rich description.

Feature 3: Popularity Score (Log-Normalised Views)

The number of views ranges from 0 to 50,000+. Such a huge range causes problems — a 50,000-view listing would completely dominate a 2,000-view one. We apply a **logarithm** to compress the scale:

```
log_views = log(1 + views) # log1p handles 0 views safely  
popularity = (log_views - min) / (max - min) # Normalise to [0,1]
```

Result: a listing with 50,000 views scores ~1.0, one with 1,000 views scores ~0.6, and one with 10 views scores ~0.2. The differences are meaningful but not extreme.

Feature 4: Recency Score (Exponential Decay)

A listing posted today is more likely to still be available than one posted 2 years ago. We model this with an **exponential decay** function:

```
recency_score = exp(-days_ago / 30)

# Examples:
# Posted today (0 days):    exp(0)    = 1.00
# Posted 1 month ago:      exp(-1)    = 0.37
# Posted 3 months ago:     exp(-3)    = 0.05
# Posted 1 year ago:       exp(-12)   ≈ 0.00
```

Feature Summary Table

Feature	Type	Formula	Purpose
_text	Text	Title + Location + price_bucket	Input to TF-IDF
price_bucket	String	pd.qcut(Price_MRU, q=5)	Semantic price label
popularity_score	Float 0-1	normalize(log1p(Views))	Proxy quality signal
recency_score	Float 0-1	exp(-days_ago / 30)	Freshness signal
price_normalized	Float 0-1	MinMaxScaler on Price_MRU	Numeric price comparison

6. The Recommender System Algorithm

A **recommender system** is an algorithm that, given a user's request, selects and ranks items from a database by how well they match. Think of Netflix recommending movies, or Amazon recommending products — same idea, applied to Mauritanian real estate.

6a. Why Content-Based Filtering?

There are three main families of recommendation algorithms. The choice depends entirely on **what data is available**:

Algorithm Family	What It Needs	Available?	Our Decision
Collaborative Filtering (User-User)	Many users + their ratings/purchases to find 'users like you'	NO — no user accounts	Cannot use
Collaborative Filtering (Item-Item)	Which items users viewed/bought together (co-occurrence patterns)	NO — no purchase/click log	Cannot use
Matrix Factorisation (SVD, ALS, NMF)	User × Item interaction matrix (ratings or implicit feedback)	NO — no interaction data	Cannot use
Neural CF / BERT4Rec	Large labelled interaction dataset + GPU compute	NO — no labels, overkill	Cannot use
Knowledge Graph	Ontology of property attributes (bedrooms, m ² , proximity to services)	PARTIAL — only views	Future work
Content-Based Filtering ✓	Item descriptions: title, price, location, views, date	YES — all available	USED HERE

Content-Based Filtering describes what the user wants (through conversation) and finds the most similar items based on their descriptions. No historical user data needed — perfect for a new platform.

6b. Algorithms We Did NOT Use — And Why

Collaborative Filtering (User-User / Item-Item)

These are the most popular recommendation algorithms used by Netflix and Amazon. They find patterns across many users: if Ali and Fatima both liked listings A, B, C, and Fatima also liked D, then Ali might like D too. **Problem:** This requires a database of users and their interaction history. Our system has no registered users, no ratings, and no purchase history. We only have listing metadata.

Matrix Factorisation (SVD, ALS, NMF)

These algorithms decompose a large User × Item matrix into smaller hidden factors (latent features). Spotify uses a variant of this. **Problem:** Without the User × Item matrix, these algorithms literally cannot be computed. They need user-item interactions as input.

Neural Networks / Deep Learning (BERT4Rec, NeuMF)

Modern deep learning models can learn complex patterns from interaction sequences.

Problems: (1) Require tens of thousands of user interaction examples to train meaningfully; (2) Require GPU infrastructure; (3) Our dataset has 1,852 items — far too small to benefit from deep learning. A simple TF-IDF outperforms a neural network on small datasets.

Sentence Transformers / Semantic Search

Models like paraphrase-multilingual-MiniLM could understand that *"maison"* and *"villa"* are semantically related. **Problem:** These models have sizes of 100MB–400MB, require more compute, and need GPU for real-time inference. For this project's scale, TF-IDF with character n-grams achieves comparable results. This is marked as a future improvement.

6c. TF-IDF — The Text Similarity Engine

TF-IDF stands for Term Frequency – Inverse Document Frequency. It is a classical NLP technique that turns text into a numerical vector, where important and rare words get higher weights.

What is TF-IDF? (For Beginners)

Imagine you have 1,000 property listings. The word 'the' appears in every single one — it carries zero information. But 'Tevragh Zeina' appears in only 150 listings — it is specific and informative. TF-IDF automatically assigns higher weight to rare, specific words:

TF (Term Frequency)	= How often does the word appear in THIS document?
IDF (Inverse Doc Freq)	= How RARE is the word across ALL documents? ($\log(N/df)$)
TF-IDF = TF × IDF	
'the' in property text	→ TF high, IDF low → score ≈ 0
'Tevragh Zeina'	→ TF varies, IDF high → score high
'villa'	→ TF varies, IDF medium → medium score

Why Character N-grams? (Not Words)

Standard TF-IDF splits text into words. But our data contains **Arabic and French** mixed together. Arabic has complex morphology — the word for 'house' has 12+ forms depending on

grammar. Instead of words, we use **character n-grams**: sequences of 2–4 characters.

```
# Word-level: 'villa' = 1 token
# Char 2-gram: 'villa' = ['vi', 'il', 'll', 'la']
# Char 3-gram: 'villa' = ['vil', 'ill', 'lla']
# Char 4-gram: 'villa' = ['vill', 'illa']

# This means 'villa', 'Villla' (typo), 'villaa' all
# share many n-grams → fuzzy matching for free!
```

Benefits: handles Arabic without a tokenizer, handles spelling variants, handles French words embedded in Arabic text. Setting: **ngram_range=(2,4), max_features=8000, sublinear_tf=True, analyzer='char_wb'**.

Cosine Similarity — How We Compare Vectors

After TF-IDF converts texts to vectors (lists of 8,000 numbers), we need to measure how similar a query vector is to each listing's vector. We use **cosine similarity**: the cosine of the angle between two vectors.

```
cosine_similarity = (A · B) / (|A| × |B|)

# Result: 0.0 = completely different
#         1.0 = identical
#         0.7 = very similar
#         0.3 = somewhat related

# Example: query='villa Teveragh Zeina'
# Listing A (villa, Teveragh Zeina) → similarity = 0.82
# Listing B (apartment, Arafat) → similarity = 0.11
```

6d. Hybrid Scoring Formula

Text similarity alone is not enough. A listing perfectly matching the keywords but priced at 10× the budget, or listed 3 years ago, is not a good recommendation. We combine four signals into a **weighted hybrid score**:

```
final_score = (0.65 × text_similarity)
              + (0.15 × price_compatibility)
              + (0.12 × popularity_score)
              + (0.08 × recency_score)
```

Component	Weight	What It Measures	Why This Weight?
Text Similarity	65%	How well the title/location/type matches the user's query	Query keywords are the PRIMARY signal — what the user asked for

Price Compatibility	15%	How well the price fits the user's budget (1 = perfect, 0 = over budget)	Budget is a HARD constraint in real estate — 2nd most important
Popularity Score	12%	How many views the listing has (log-normalised)	Proxy signal: popular listings are often better quality or priced
Recency Score	8%	How recently the listing was posted (exponential decay)	Fresh listings more likely still available — tie-breaker

■ *The weights were chosen based on domain knowledge: in real estate, WHAT you want and HOW MUCH you can pay dominate everything else. These weights can be tuned with user feedback data in the future.*

7. Voice to Text — How The Microphone Works

When the user clicks the microphone button in the app, their voice is recorded as a **WAV audio file** (raw audio bytes). These bytes are then sent to Google's Speech Recognition service via the **SpeechRecognition** Python library. This is handled in **src/voice.py** → **transcribe_audio_bytes()**.

Step-by-Step Process

- User presses mic button → `audio_recorder_streamlit` captures audio at 16,000 Hz sample rate
- Audio bytes are written to a temporary WAV file on disk
- The `SpeechRecognition` library loads the WAV file as an `AudioFile`
- We try 3 languages in order: English (en-US) → Moroccan Arabic (ar-MA) → French (fr-FR)
- The first language that returns a non-empty result is used
- The temporary file is deleted and the transcribed text string is returned
- If all 3 languages fail, we return `None` and show a warning to the user

```
def transcribe_audio_bytes(audio_bytes: bytes) -> str | None:
    # Save audio bytes to temp WAV file
    with tempfile.NamedTemporaryFile(suffix='.wav') as f:
        f.write(audio_bytes)
    # Load and try multiple languages
    with sr.AudioFile(tmp) as source:
        audio = recognizer.record(source)
        for lang in ('en-US', 'ar-MA', 'fr-FR'):
            text = recognizer.recognize_google(audio, language=lang)
            if text: return text # Return first successful result
```

Audio Deduplication — `audio_hash()`

Streamlit re-runs the entire Python script on every user interaction. Without a deduplication check, the same audio clip would be re-transcribed on every rerun. We solve this with an **MD5 hash**: we compute a fingerprint of the audio bytes and only process it if the hash is different from the last processed audio.

Wake Word Detection

The app recognises wake words like *"Hey Jarvis"*, *"Hi Jarvis"*. If the transcribed text matches exactly, the AI replies with *"At your service, sir"* without calling the full recommender pipeline. If the text starts with a wake prefix, the prefix is stripped and only the actual query is sent to Mistral.

Older Voice Chatbot Prototype — `chatbot.py`

The file **`chatbot.py`** is the original command-line prototype built before the Streamlit app. It uses:

- `speech_recognition` for microphone input (real-time stream, not file upload)
- `gTTS` (Google Text-to-Speech) for audio output
- `pygame` to play the generated MP3 audio
- Mistral API for the chat conversation (no recommender, no property search)

This prototype proved the voice + LLM pipeline worked before integrating the recommender system and building the Streamlit UI.

8. Text to Speech — How The AI Speaks Back

After the AI generates a response text, we convert it back to audio so the user can hear it. This is handled in `src/voice.py` → `text_to_speech_bytes()`.

Two TTS Engines Used in This Project

Engine	Library	Where Used	Voice Quality	Language
Google TTS	gTTS	chatbot.py (prototype)	Standard, robotic	English
Microsoft Neural TTS	edge-tts	src/voice.py (production)	Neural, JARVIS-like	EN + AR

The Production TTS: Microsoft edge-tts

The production app uses **edge-tts**, which connects to Microsoft Azure's Neural TTS service for free via the Edge browser API. It produces high-quality, natural-sounding speech — specifically the voice **en-GB-RyanNeural** (British male), chosen to sound like JARVIS from Iron Man.

```
JARVIS_VOICE = 'en-GB-RyanNeural' # British male neural voice
ARABIC_VOICE = 'ar-EG-ShakirNeural' # Egyptian Arabic male
JARVIS_RATE = '-5%' # Slightly slower, more authoritative

# Language detection: if >25% of characters are Arabic,
# switch to the Arabic voice automatically
def _is_arabic(text):
    arabic_chars = sum(1 for c in text if '\u0600' <= c <= '\u06ff')
    return arabic_chars > len(text) * 0.25
```

Why Async? — asyncio

The edge-tts library is **asynchronous** — it uses Python's `async/await` syntax to fetch audio without blocking. Since Streamlit runs synchronously, we create a new event loop, run the `async` function to completion, then close the loop. The result is raw MP3 bytes that we encode as base64 and inject into JavaScript for autoplay.

Autoplay in the Browser

Web browsers block audio autoplay by default for security. We work around this by injecting a hidden JavaScript snippet that creates an Audio object with a base64-encoded MP3 data URI

and plays it immediately. The played set in session state ensures each message plays only once.

9. The AI Brain — Mistral Large & JARVIS

Mistral Large is a large language model (LLM) made by the French AI company Mistral AI. It is comparable to GPT-4 in capability. We use it for two distinct purposes in this project.

Purpose 1: Conversational Understanding (chat_turn)

Mistral acts as a conversational agent with a very specific **system prompt** that gives it the JARVIS personality. The conversation follows a structured flow: gather property type → budget → neighborhood → special requirements → confirm → trigger search.

The key innovation is the **<SEARCH> tag**: when Mistral has collected enough information, it appends a JSON block at the end of its response:

```
{
  "keywords": "villa garden",
  "location": "Tevragh Zeina",
  "price_min": null,
  "price_max": 2000000}
}
```

Python then extracts this JSON using regex, parses it, and passes the parameters to the recommender. The **<SEARCH>** tag is stripped from the displayed text — the user only sees the JARVIS dialogue.

Purpose 2: Result Formatting (format_results)

After the recommender returns matching listings, we pass them back to Mistral with a prompt asking it to write a professional JARVIS-style briefing. This transforms raw data into a polished response like: *"I've located three properties matching your criteria, sir. The most promising is a 5-room villa in Tevragh Zeina at 1,800,000 MRU — well within your budget and with 842 views, indicating strong market interest, sir."*

JARVIS Personality Rules

- Always addresses user as 'sir' — at least once per message
- Opens with: 'Yes, sir', 'Of course, sir', 'Right away, sir', etc.
- Formal British English — never casual ('Hey!', 'Sure!', 'Awesome!')
- Maximum 3 sentences per reply — concise and precise
- References data like an AI: 'I've cross-referenced the database', 'The analysis indicates'
- One question per message — structured conversation flow

Multi-Turn Conversation Memory

The full conversation history is stored in **st.session_state.conv** as a list of messages: `[{'role': 'user', 'content': '...'}, {'role': 'assistant', 'content': '...'}]`. This entire history is sent with every API call, so Mistral always has context. The display history (**st.session_state.display**) is separate — it also stores audio bytes and property card data for rendering.

10. The Web App — Streamlit UI

Streamlit is a Python library that turns Python scripts into interactive web apps with no JavaScript or HTML needed. The main app is **app.py**.

Three Tabs

Tab	What It Shows
■ AI Assistant	The main chatbot: mic button, chat history, property cards with images/links/scores
■ Data Explorer	Interactive charts: price distribution, top neighborhoods, views vs. price scatter, raw data table
■ Evaluation	Run Precision@K evaluation on 8 test queries, see per-query scores, system architecture summary

Sidebar Filters

The sidebar shows live dataset statistics and allows the user to set:

- Min / Max Price (MRU) — applied to recommender queries
- Neighborhood filter — dropdown of all available locations
- Results to show — slider from 3 to 10
- Available only checkbox — filter out sold listings
- Reset Conversation button — clears all session state

Property Cards

Each recommended property is rendered as a rich card inside an expander. The card shows: listing image (or placeholder), title, price in MRU, location, view count, availability badge, hybrid score percentage, price bucket badge, and a link button to the original voursa.com listing.

Caching Strategy

Two Streamlit cache decorators prevent re-loading data on every interaction:

- `@st.cache_data` — caches the cleaned DataFrame (`load_and_clean`) across sessions
- `@st.cache_resource` — caches the fitted Recommender object (expensive: builds TF-IDF matrix)

11. Evaluation — How We Measure Quality

Without user feedback (ratings, clicks), formal evaluation is challenging. We use a **Precision@K proxy metric**.

What is Precision@K?

For a given query, we retrieve the top K results. Precision@K is the fraction of those K results that are considered 'relevant'. In our case, we define a result as relevant if its **hybrid score > 0.1** (meaning the text similarity is non-zero — at least some keyword matched).

```
Precision@K = (number of relevant results in top K) / K
```

```
Example: query='villa Tévragh Zeina', K=5
```

```
→ 4 out of 5 results have score > 0.1
```

```
→ Precision@5 = 4/5 = 0.80 (80%)
```

Test Query Set

Query	What It Tests
villa Tévragh Zeina luxury	High-end specific search
cheap house Arafat	Budget + location
apartment Ksar affordable	Type + location + price tier
land terrain constructible	Mixed language (FR/EN)
commercial space office	Non-residential property type
maison dar vendre Sebkha	French/Arabic mix
duplex Dar Naim	Specific property type + location
studio apartment Teyarett	Small apartment search

Limitations of Current Evaluation

This proxy metric is better than nothing, but it has limitations:

- A result with score > 0.1 might still be irrelevant to a human evaluator
- We cannot compute NDCG or MAP without human-labelled relevance judgements
- The test set is small (8 queries) — may not represent all real user needs
- Evaluation is not cross-validated — no train/test split on the dataset

Better evaluation would require: collecting real user click data, asking domain experts to rate results, or running A/B tests comparing different algorithm configurations.

12. Future Improvements

Improvement	Impact	Difficulty
Multilingual sentence-transformers (paraphrase-multilingual-MiniLM)	Better Arabic semantic matching — understands that 'villa' $\approx$ 'dar' $\approx$ '██████'	Medium — needs GPU or API
User session logging → implicit CF	Unlock collaborative filtering signals from click/view patterns	High — needs auth + DB
Price-per-m ² feature	Fairer comparison across property sizes if area data added	Low — data collection needed
Re-ranking with LLM based on full conversation	Higher precision for complex requirements	Medium — more API calls
Map visualisation (folium / pydeck)	Show properties on an interactive map of Nouakchott	Low — library integration
Saved searches and alerts	Notify users when new matching listings appear	High — needs backend
Rigorous evaluation (NDCG, MAP)	Real accuracy measurement for algorithm comparison	Medium — needs human labels
FAISS vector index for speed	Sub-millisecond search on 100K+ listings	Low — library swap

13. Glossary — Key Terms Explained

Term	Definition
Algorithm	A set of rules or instructions that a computer follows to solve a problem.
API	Application Programming Interface — a way for two programs to communicate. We call Mistral's API to get AI responses.
Async / asyncio	A Python programming style where tasks can run concurrently without blocking each other, like downloading a file while doing something else.
Chatbot	A program that has a text or voice conversation with a human, mimicking human dialogue.
Collaborative Filtering	A recommendation method that uses patterns from many users ('people like you also liked...').
Content-Based Filtering	A recommendation method that compares the content/description of items to find similar ones.
Cosine Similarity	A way to measure how similar two vectors are, based on the angle between them. 1.0 = identical, 0.0 = completely different.
CSV	Comma-Separated Values — a simple spreadsheet format where each row is a line and columns are separated by commas.
Exponential Decay	A mathematical curve where a value decreases rapidly at first then more slowly. Used for recency scoring.
Feature Engineering	The process of creating new useful columns (features) from raw data to improve algorithm performance.
Hybrid Score	A combined score that blends multiple signals (text, price, popularity, recency) with different weights.
LLM	Large Language Model — an AI trained on billions of text examples, capable of understanding and generating natural language (e.g., Mistral, GPT-4).
Log / log1p	Logarithm — a mathematical operation that compresses large ranges. $\log(10)=1$, $\log(1000)=3$, $\log(1,000,000)=6$.
MD5 Hash	A fingerprint of data — the same input always gives the same hash, different inputs give different hashes. Used to detect duplicate audio.
Min-Max Scaling	Normalising values to the range [0, 1] by: $(\text{value} - \text{min}) / (\text{max} - \text{min})$.
N-gram	A sequence of N consecutive characters or words. Char 3-gram of 'villa' = ['vil', 'ill', 'lla'].
NLP	Natural Language Processing — the field of AI dealing with human language (reading, writing, speaking, understanding).
Outlier	A data point that is far from the typical range (e.g., a house listed at 1 billion MRU — likely a typo).

Percentile	A value below which a certain percentage of observations fall. 99th percentile = value that 99% of data is below.
Precision@K	The fraction of the top K retrieved results that are actually relevant.
Quantile	Dividing data into equal-sized groups. Quintiles (q=5) divide into 5 groups of 20% each.
Regex	Regular Expression — a pattern for matching text. <code>r'^\d'</code> matches any non-digit character.
Recommender System	An algorithm that suggests relevant items to users based on their preferences or item features.
Session State	In Streamlit, a dictionary that persists data across reruns of the app for a single user session.
Speech Recognition	The technology that converts spoken audio into text. We use Google's service.
STT	Speech-to-Text — converting audio/voice into written text.
Streamlit	A Python library for building interactive web apps without needing JavaScript or HTML.
TF-IDF	Term Frequency–Inverse Document Frequency — a way to convert text to numbers, giving higher weight to rare, specific words.
TTS	Text-to-Speech — converting written text into spoken audio.
Vector	A list of numbers. TF-IDF turns text into a vector of 8,000 numbers representing word importances.
WAV	A raw audio file format. Uncompressed — large but high quality. Used internally before sending to Speech Recognition.

End of Documentation

Immo.AI — Mauritania Real Estate Recommendation System

Built with Content-Based Filtering · TF-IDF · Mistral AI · Google STT · Microsoft TTS

June 2026 · aliywedad · 22086@supnum.mr
